Real-world (one reusable session) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `session_reuse_practice.py`: headers passed by hand to every call are a typo waiting to happen — one call in five forgets the `Authorization` header, and nothing tells you until a request mysteriously gets rejected. One shared `Session`, configured once, makes that mistake structurally impossible. **If not:** `http_client.py` in the Build Task would open a new connection and re-specify headers on every single call, slower and more fragile than it needs to be.

Solution

```
# session_reuse_practice.py
import requests

def make_authenticated_session(token: str) -> requests.Session:
    # why: one shared Session with the header set once, instead of passing
    # the same headers dict to every call by hand where it can drift or get
    # missed.
    # when: call this once, at the start of the program — treat the
    # returned session's headers as read-only after this point.
    session = requests.Session()
    session.headers.update({"Authorization": f"Bearer {token}"})
    return session

def main() -> None:
    session = make_authenticated_session("fake-token")

    urls = [
        "https://api.github.com",
        "https://api.github.com/zen",
        "https://api.github.com/octocat",
    ]

    for url in urls:
        # how: session.get(...), not requests.get(...) — this is what makes
        # the Authorization header go out automatically on every call.
        response = session.get(url)
        print(f"{url} -> {response.status_code}")
```

```
    # how: response.request.headers reads back what was actually sent over
    # the wire, not just what's configured on the session object.
    sent_header = response.request.headers.get("Authorization")
    print(f"header actually sent on the last request: {sent_header}")

if __name__ == "__main__":
    main()
```

Expected output:

```
https://api.github.com -> 200
https://api.github.com/zen -> 200
https://api.github.com/octocat -> 200
header actually sent on the last request: Bearer fake-token
```

`response.request.headers` reads back the *actual* request object `requests` sent, not the session's own dict — confirming the header genuinely went out over the wire, not just that it's set somewhere in Python.